ARE 212 Section 7:
GLS and MLE

Fiona Burlig

March 3, 2016

This week, we'll start by briefly covering generalized least squares (and using our newfound `ggplot()` skills to do so), and then we'll spend the remainder of the time talking about maximum likelihood estimation in `R`.[1]

Anything but ordinary

That song was my *jam* in middle school.[2] We've done a lot of work up to this point using the applied econometrician's workhorse, *ordinary* least squares (OLS). But it turns out there are also extraordinary least squares to be found in the world. We'll take a look at generalized least squares (GLS) today - and in particular, weighted least squares (WLS), which, like OLS, is a specific case of GLS.[3] Replication is a good thing in science in general - you should always aim to make your results replicable - so we'll start off by trying to replicate Max's figures 2.6 and 2.7 (albeit more attractively).[4] Before we do that, though, let's remind ourselves exactly what GLS and WLS do:

Something old, something new, something borrowed, something BLUE

When we were talking about OLS, we assumed that our errors are spherical, or that the errors are heteroskedastic and uncorrelated. In math:

$$\mathbb{E}[\varepsilon\varepsilon'|\mathbf{X}] = \sigma^2\mathbf{I}_n$$

Under this assumption, we showed earlier that OLS is BLUE.[5] But what if we relax this assumption? OLS will still be unbiased (and you should re-think your t-test and F-test), but it won't be BLUE any longer. All is not lost, however: it turns out that we can write down a more general assumption, and come up with a new something BLUE. Our generalized assumption looks like this:

$$\mathbb{E}[\varepsilon\varepsilon'|\mathbf{X}] = \sigma^2\mathbf{\Omega}(\mathbf{X})$$

If $\mathbf{\Omega}(\mathbf{X})$ is known and positive-definite, it can be shown that GLS is BLUE.[6] Because GLS is BLUE in this case and OLS isn't (necessarily), it'd behoove us to use GLS when we can. Why? Since GLS is the more efficient

[1] As an aside: my jokes last week were sub-par. Apologies. Hopefully this week makes up for it.

[2] Who are we kidding? It's still my jam.

[3] Actually, all of these things can be expressed as a special case of GMM - but that's a story for another day. If you're interested in this, go talk to Ethan, or take Pat Kline's ECON 244 course, which carries the official ARE 212 Section stamp of approval.

[4] Shots fired.

[5] Whoo!

[6] See Max's notes. Ain't nobody in section got time for that proof.

estimator in this case, it'll give us smaller standard errors. Everyone loves small standard errors. So let's play with a special case of GLS - WLS.

Been weighting all my life

Let's impose a little bit of structure on our $\mathbf{\Omega}(\mathbf{X})$ matrix (but not enough to get back to our OLS-is-BLUE happy place). In particular, suppose that there's no correlation across observations. This makes $\mathbf{\Omega}(\mathbf{X})$ diagonal.[7] Letting $\omega_i(X)$ be the ith diagonal element of $\mathbf{\Omega}(\mathbf{X})$, we can write our new assumption as:

$$\mathbb{E}[\varepsilon\varepsilon'|\mathbf{X}] = \sigma^2\boldsymbol{\omega}_i(\mathbf{X})$$

Given this assumption, we can weight each of our observations by $w_i = \frac{1}{\sqrt{\omega_i(X)}}$ and run OLS to recover the WLS result. Let's give this a try. To test the efficiency of the WLS estimator, we'll start by creating a population. Then, we'll draw a bunch of random samples from that population, run both weighted and unweighted regressions on them, and plot our resulting $\hat{\beta}$s from WLS vs OLS.

To implement this in practice, we'll first load our usual packages and grab our (small) OLS function from last week. Since we'll be randomly generating data today, let's also set our seed. Finally, we'll bring in our theme options for `ggplot2`.

```
########## PACKAGES
library(lfe)
library(dplyr)
library(readr)
library(ggplot2)

########## FUNCTIONS
tbldfGrabber <- function(data, varnames) {
  matrixObject <- data %>%
    select_(.dots = varnames) %>%
    as.matrix()
}
as.tbl_df <- function(data) {
  dataset <- as.data.frame(data) %>%
    tbl_df()
}

smallOLS <- function(data, y, X) {
  ydata <- tbldfGrabber(data, y)
  xdata <- tbldfGrabber(data, X)
  betahat <- solve(t(xdata) %*% xdata) %*% t(xdata) %*% ydata
  return(betahat)
}

########## RANDOMIZATION SEED
```

[7] Thinking about why will help you understand standard errors later.

```r
set.seed(12345)


########## GGPLOT SETUP
myThemeStuff <- theme(panel.background = element_rect(fill = NA),
        panel.border = element_rect(fill = NA, color = "black"),
        panel.grid.major = element_blank(),
        panel.grid.minor = element_blank(),
        axis.ticks = element_line(color = "gray5"),
        axis.text = element_text(color = "black", size = 10),
        axis.title = element_text(color = "black", size = 12),
        legend.key = element_blank()
        )
```

Okay, great. We'll use the same DGP as Max for this exercise, $y_i = \alpha + x_i\beta + \varepsilon_i$. We'll set $X \sim U[0, 2000]$, $\varepsilon \sim N(0, 400 \cdot \frac{1}{100}x_i^2)$, $\alpha = 0.5$, and $\beta = 1.5$. Notice that we've (implicitly) set $\sigma^2 = 400$ and $w_i = \frac{1}{100}x_i^2$. We're going to implement WLS by drawing a bunch of samples from a population of 100,000 units we set up according to the above specifications, so let's start by generating the population dataset.

```r
popN <- 100000
popSigma2 <- 400
popX <- runif(popN, min = 0, max = 2000)
popWeight <- (1/100) * popX^2
popVariance <- popSigma2 * popWeight
# note that rnorm takes the SD, not the variance, as its argument
#  hence the sqrt() here.
popEpsilon <- rnorm(popN, mean = 0, sd = sqrt(popVariance))
popY <- 0.5 + popX * 1.5 + popEpsilon

# package this guy into a tbl_df:
myPopulation <- cbind(popY, 1, popX, popWeight) %>%
  as.tbl_df()
names(myPopulation) <- c("y", "ones", "X", "weight")
```

Population is ready to go. Now let's grab a sample. Like Max, we'll draw samples of size 1,000 from our population, and run regressions on those samples. We'll set this up in a function so that we can arbitrarily set the number of samples we'd like to draw.

```r
randomLSSample <- function(sampSi, data, y, X, weight) {
  sampledData <- sample_n(data, size = sampSi)
  sampledY <- tbldfGrabber(sampledData, y)
  sampledX <- tbldfGrabber(sampledData, X)
  sampledWeight <- tbldfGrabber(sampledData, weight)
  cMat <- diag(1/sqrt(sampledWeight[,1]))

  yWt <- cMat %*% sampledY
  xWt <- cMat %*% sampledX
```

```
  weightedData <- cbind(yWt, xWt) %>%
    as.tbl_df()
  names(weightedData) <- c(y, X)

  bOLS <- smallOLS(sampledData, y, X)[2]
  bWLS <- smallOLS(weightedData, y, X)[2]

  output <- cbind(bOLS, bWLS)
  return(output)
}
```

Because this can take a little while, I want to run it in parallel. If you're not up to speed on parallel processing, skip down below.

```
library(parallel)
cl <- makeCluster(11)

clusterExport(cl, "myPopulation")

clusterExport(cl, "as.tbl_df")
clusterExport(cl, "tbldfGrabber")
clusterExport(cl, "smallOLS")
clusterExport(cl, "randomLSSample")
clusterEvalQ(cl, library(dplyr))
clusterSetRNGStream(cl, 12345)
```

And off we go:

```
figureData <- parSapply(cl, 1:1000, function(x) {
  randomLSSample(1000, myPopulation, "y", c("ones", "X"), "weight")
})
```

Normally, you should stop your clusters when you're done with them. We'll use them later on, so we'll leave them open. Also, we should really be doing 100,000 draws - but that will take too long in class (even parallelized), so we'll do 1,000 instead.

Here's the non-parallel version:

```
figureData <- sapply(1:1000, function(x) {
  randomLSSample(1000, myPopulation, "y", c("ones", "X"), "weight")
})
```

This came out in two rows and many columns, which I find kind of annoying. Let's transform it, put it into `tbl_df` format, and name things nicely:

```
figureData <- t(figureData) %>%
  as.tbl_df()
names(figureData) <- c("OLS", "WLS")
```

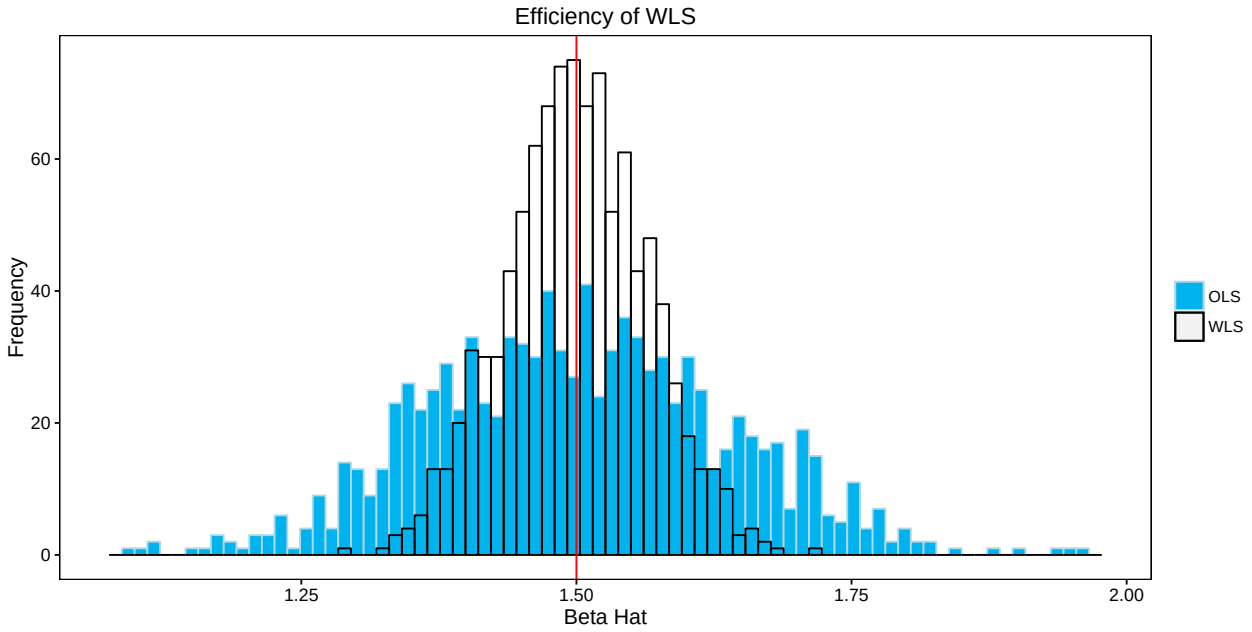
Much better. Now we can use our `ggplot2` skills from last week to replicate Max's figures:

```
library(ggplot2)

myThemeStuff <- theme(panel.background = element_rect(fill = NA),
        panel.border = element_rect(fill = NA, color = "black"),
        panel.grid.major = element_blank(),
        panel.grid.minor = element_blank(),
        axis.ticks = element_line(color = "gray5"),
        axis.text = element_text(color = "black", size = 10),
        axis.title = element_text(color = "black", size = 12))

maxHistogram <- ggplot(figureData) +
  # label the colors so that we can have a legend
  geom_histogram(aes(x = OLS, color = "OLS", fill = "OLS"),  bins = 75,
                 position = "identity") +
  geom_histogram(aes(x = WLS, color = "WLS", fill = "WLS"),  bins = 75,
                 position = "identity") +
  geom_vline(xintercept = 1.5, color = "red") +
  ggtitle("Efficiency of WLS") +
  xlab("Beta Hat") + ylab("Frequency") +
  myThemeStuff +
  scale_fill_manual(values = c("deepskyblue2", "NA")) +
  scale_color_manual(values = c("lightblue", "black")) +
  theme(legend.title = element_blank())
```

```
maxHistogram
```



It should be obvious from this figure that WLS is more efficient than OLS, given that we have the correct model

for $\mathbf{\Omega}(\mathbf{X})$. It's a little hard to see, since the X-axis is so wide here, but we can prove to ourselves that both OLS and WLS are unbiased:

```
summarize(figureData, mean(OLS))

## Source: local data frame [1 x 1]
##
##   mean(OLS)
##       (dbl)
## 1  1.501462

summarize(figureData, mean(WLS))

## Source: local data frame [1 x 1]
##
##   mean(WLS)
##       (dbl)
## 1  1.499722
```

Excellent. We're now officially GLS-ninjas. Every time you run a regression from now on, you're going to use GLS rather than OLS, right? Everybody loves efficiency! ...well, sort of. It turns out that the assumption that we know $\mathbf{\Omega}(\mathbf{X})$ perfectly is a very strong assumption. What happens if we misspecify the weighting scheme?

```
wrongGLS <- function(sampSi, data, y, X, weight) {
  sampledData <- sample_n(data, size = sampSi)
  sampledY <- tbldfGrabber(sampledData, y)
  sampledX <- tbldfGrabber(sampledData, X)
  sampledWeight <- tbldfGrabber(sampledData, weight)

  # NOTE THE ERROR WE'RE MAKING HERE: WE JUST REMOVED THE SQUARE ROOT. BAD NEWS BEARS.
  cMat <- diag(1/(sampledWeight[,1]))

  yWt <- cMat %*% sampledY
  xWt <- cMat %*% sampledX

  weightedData <- cbind(yWt, xWt) %>%
    as.tbl_df()
  names(weightedData) <- c(y, X)

  bOLS <- smallOLS(sampledData, y, X)[2]
  bWLS <- smallOLS(weightedData, y, X)[2]

  output <- cbind(bOLS, bWLS)
  return(output)
}
```

For parallel folks, let's export our new function to the cluster, and run the function. We're finished with the clusters now, so we'll go ahead and close them.

```
clusterExport(cl, "wrongGLS")
wrongFigureData <- parSapply(cl, 1:1000, function(x) {
  wrongGLS(1000, myPopulation, "y", c("ones", "X"), "weight")
})
stopCluster(cl)
```

Again, here's the non-parallel version:

```
wrongFigureData <- sapply(1:1000, function(x) {
  wrongGLS(1000, myPopulation, "y", c("ones", "X"), "weight")
})
```

We'll again format our data more nicely.

```
wrongFigureData <- t(wrongFigureData) %>%
  as.tbl_df()
names(wrongFigureData) <- c("OLS", "WLS")
```

And as above, make our beautiful figure.

```
wrongMaxHistogram <- ggplot(wrongFigureData) +
  # label the colors so that we can have a legend
  geom_histogram(aes(x = OLS, color = "OLS", fill = "OLS"),  bins = 75, position = "identity") +
  geom_histogram(aes(x = WLS, color = "WLS", fill = "WLS"),  bins = 75, position = "identity") +
  geom_vline(xintercept = 1.5, color = "red") +
  ggtitle("Efficiency of WLS") +
  xlab("Beta Hat") + ylab("Frequency") +
  myThemeStuff +
  scale_fill_manual(values = c("deepskyblue2", "NA")) +
  scale_color_manual(values = c("lightblue", "black")) +
  theme(legend.title = element_blank())
```

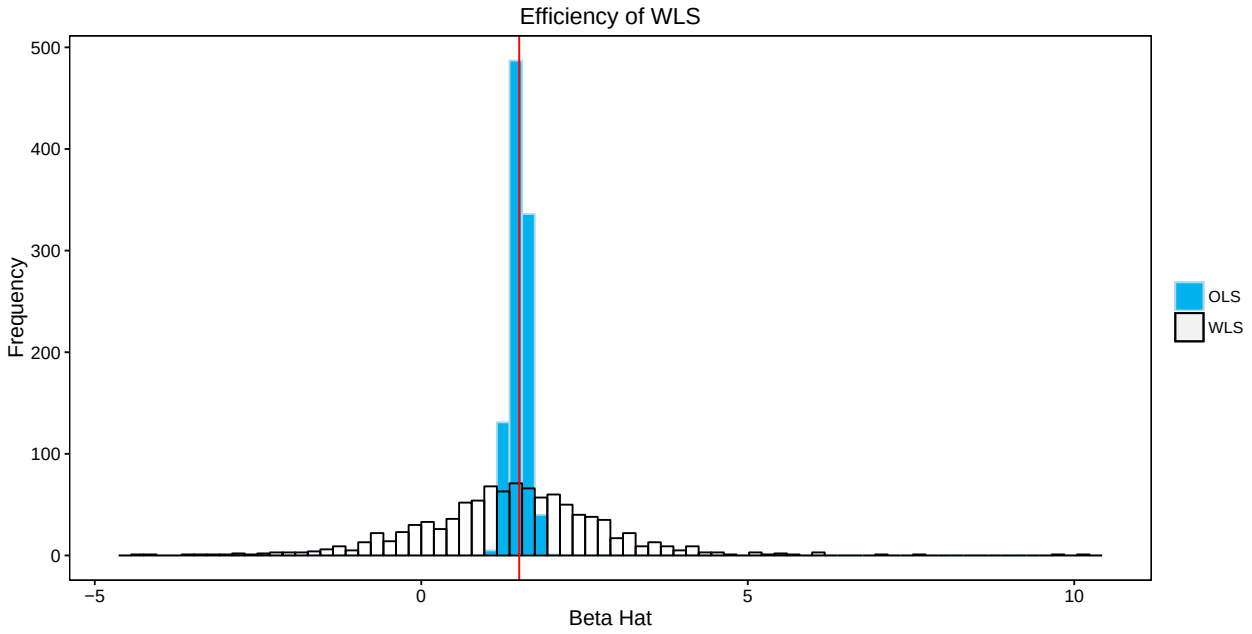
What?! This looks different. Notice that two striking things happened: first, WLS is now dramatically *less* efficient than OLS, and second, the X axis changed - not only is WLS doing badly than OLS, it's doing really, really terribly. Check out the summary stats to see what's happened:

```
summary(figureData)
```

```
##       OLS             WLS
##  Min.   :1.093   Min.   :1.295
##  1st Qu.:1.405   1st Qu.:1.457
##  Median :1.497   Median :1.498
##  Mean   :1.501   Mean   :1.500
##  3rd Qu.:1.594   3rd Qu.:1.544
##  Max.   :1.960   Max.   :1.715
```

```
summary(wrongFigureData)
```

```
wrongMaxHistogram
```




```
##       OLS              WLS
##  Min.   :1.111   Min.   :-4.3494
##  1st Qu.:1.403   1st Qu.: 0.6192
##  Median :1.501   Median : 1.4024
##  Mean   :1.501   Mean   : 1.3822
##  3rd Qu.:1.596   3rd Qu.: 2.1769
##  Max.   :1.950   Max.   :10.1333
```

So this is one cautionary tale as to why you shouldn't use GLS if you don't actually know $\mathbf{\Omega}(\mathbf{X})$ - and you'll basically never know $\mathbf{\Omega}(\mathbf{X})$, so you'll almost never actually implement GLS in real life. Even if you do, referees will be suspicious and make you implement OLS anyway. For fun (read: in your copious spare time), try to construct an example in which GLS gives you a *narrower* distribution of $\hat{\beta}$s than it should when you misspecify the weighting matrix - this is the really concerning case, because you'll think you know $\hat{\beta}$ with more certainty than you actually do. A little piece of life advice: always be a humble econometrician, and assume that you know as little as possible.[8]

Just briefly, we can also use our favorite canned routine to run WLS[9]:

```
(cannedWLS <- felm(data = myPopulation, y  ~ X, weights = 1/myPopulation$weight))

## (Intercept)           X
##      0.4777      1.4999
```

[8] Worked for me so far. Someone once told me that the best part of getting your PhD is learning to understand just how little you know. And I don't even have a PhD yet. So I'm not even a card-carrying little-knowledge-knower. Doesn't that make you feel great about spending your early Thursday mornings with me?

[9] Not sure why you have to specify the dataset again for the weights. But this seems to be the case.

Max-imum likelihood

We're going to take a brief Max-style foray into maximum likelihood estimation (MLE)[10], with the aim of making you realize that, just like in R, there are often multiple ways of implementing econometric models. We'll start with a quick discussion of what MLE does, and then think about how we can use MLE to fit a linear regression model.

So, what's the idea behind MLE? In short, suppose we observe a vector of data, $\mathbf{z}$, which we'll assume was drawn from a distribution $f(\mathbf{z}, \boldsymbol{\theta})$, where $\boldsymbol{\theta}$ is a vector of parameters that describe the distribution.[11] Given a family of distributions (Uniform, Normal, Binomial, Poisson, etc), our goal is to estimate its parameters by finding the $\hat{\boldsymbol{\theta}}$ that best fits the data that we observe. Confusing? Probably. Let's use an example to try to see what's going on.

All about the Washingtons

Suppose your friend offers you a betting opportunity, based on a coin flip. Before you agree, you of course want to know if her coin is rigged. We can use maximum likelihood estimation to figure this out!

The outcome of a coin flip is either 1 or 0, and can be described by a Bernoulli distribution: the flip will be 1 with probability p, and 0 with probability $1 - p$. p fully characterizes the distribution, so in our new notation, $\theta \equiv p$. We can estimate $\hat{\theta} \equiv \hat{p}$ using MLE. We'll start by "observing" a long series of coin flips (we're cheating a little bit here - we have to choose a "true" p value to set up our simulation):

```
set.seed(12345)

p <- 0.75
# the bernoulli distibution is a special case of the binomial
obs <- rbinom(n = 1000, size = 1, prob = p)

head(obs)

## [1] 1 0 0 0 1 1
```

We want to maximize the probability of observing this series of flips, given that each flip is distributed Bernoulli with probability p, by choosing p. We do this by maximizing a likelihood function, which is just the joint probability of observing these data, given that the flips are IID, given that they're distributed Bernoulli, and given some p. (Note that I basically just repeated myself.) The Bernoulli likelihood function looks like this:

$$L(p) = \Pi_{i=1}^{1000} p^{x_i}(1-p)^{1-x_i}$$

where x_i is the outcome of any given flip (equivalent to the entries in our obs vector). All MLE is doing is choosing p to maximize $L(p)$ - standard economist stuff.

If you're doing this analytically, it's often helpful to take logs (so that that ugly product sign becomes a summation sign), and generate the log-likelihood function:

$$l(p) = \sum_{i=1}^{1000} x_i \ln(p) + (1 - x_i) \ln(1-p)$$

[10] More acronyms! Exactly what this class needs. Also, apologies for the lamest section title ever.
[11] For example, a Normal distribution is described by its mean and its variance.

It should be unsurprising to you by now that we can write the likelihood function and the log-likelihood function up in R:

```
like <- function(p, x){
  prod(p^x * (1-p)^(1-x))
}

logLike <- function(p, x) {
  sum(x * log(p) + (1-x) * log(1-p))
}
```

Cool. Now we'll try to maximize the value of this function (either one) by feeding it our observed vector of coin flips, and by choosing different values of p. We can be a little clever about our choice of p - we know that the probability of a coin flip turning up "heads" must be between 0 and 1 - so p must also be between 0 and 1. Let's use `sapply()` to calculate the values of the likelihood function and log likelihood function at a variety of possible p's between 0 and 1:

```
# this creates a vector between 0 and 1, with increments of 0.001
pChoices <- seq(0, 1, 0.001)

likeEst <- sapply(pChoices, like, x = obs)
logLikeEst <- sapply(pChoices, logLike, x = obs)
```

Let's quickly plot the likelihood function:

```
mleData <- cbind(pChoices, likeEst, logLikeEst) %>%
  as.tbl_df()

likeFig <- ggplot(data = mleData, aes(x = pChoices, y = likeEst)) +
  geom_line(color = 'deepskyblue2') +
  myThemeStuff +
  ggtitle("Likelihood Function: Coin Flips") +
  xlab("Hypothesized p") + ylab("Likelihood")
```

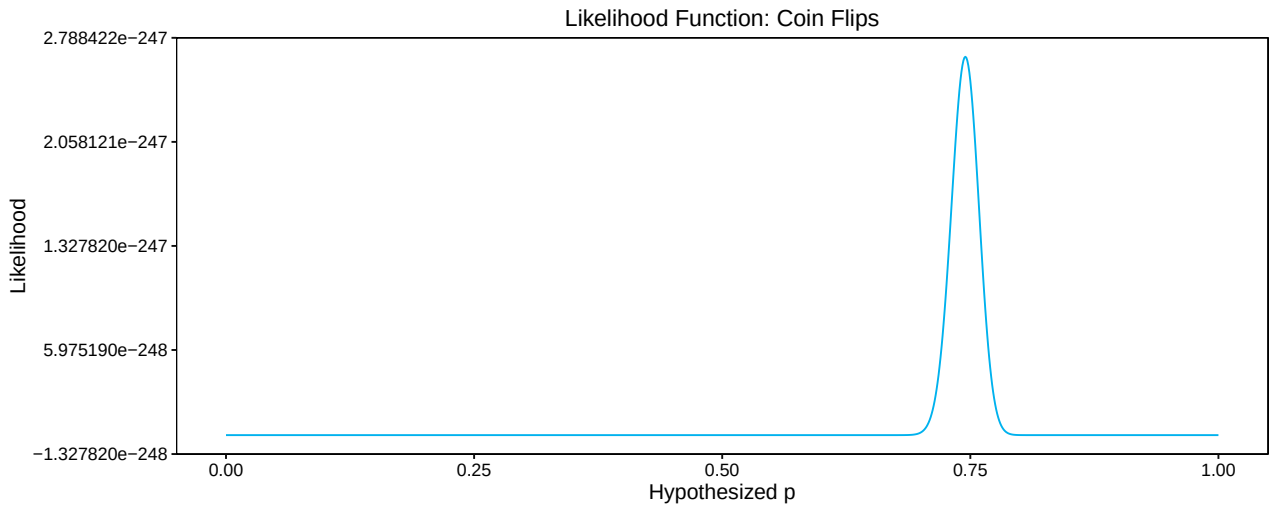
Cool - the likelihood function is obviously maximized right around $p = 0.75$, which makes sense. We can also plot the log likelihood function. Since taking logs is a monotonic transformation, this should yield the same p:

```
logLikeFig <- ggplot(data = mleData, aes(x = pChoices, y = logLikeEst)) +
  geom_line(color = 'deepskyblue2') +
  myThemeStuff +
  ggtitle("Likelihood Function: Coin Flips") +
  xlab("Hypothesized p") + ylab("Log likelihood")
```

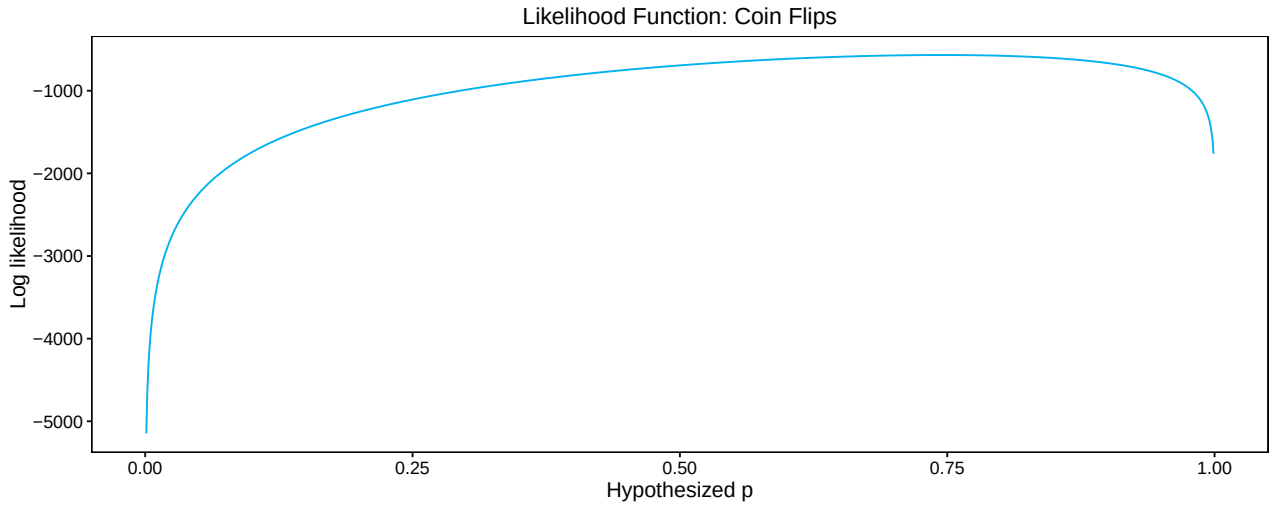
Now you see why we didn't try to plot these two on the same axis. This isnt quite as clear, but it does look like our log-likelihood function is also maximized right around $p = 0.75$.

Of course, it's hard to write a job market paper based around squinting and pointing at a figure. So let's actually be serious about this and use an optimization routine to numerically calculate p:

```
likeFig
```




```
logLikeFig
```




```
#optimize takes a function f, and searches across the interval to
#  maximize the fxn WRT its first argument

mleOptim <- optimize(f = like,
                             # set our x value
                             x = obs,
                             # sets the interval
                             interval = c(0,1),
                             # tells optimize to maximize rather than minimize
                             maximum = TRUE
                             )
mleOptim
```

```
## $maximum
## [1] 0.7449998
##
## $objective
## [1] 2.65564e-247
```

That optimized value looks pretty damn good. Let's try the log likelihood:

```
#optimize takes a function f, and searches across the interval to
#  maximize the fxn WRT its first argument

mleLogOptim <- optimize(f = logLike,
                        # set our x value
                        x = obs,
                        # sets the interval
                        interval = c(0,1),
                        # tells optimize to maximize rather than minimize
                        maximum = TRUE
                        )
mleLogOptim

## $maximum
## [1] 0.7449872
##
## $objective
## [1] -567.7618
```

Yessss. As an aside: it's often useful to use the log-likelihood rather than the likelihood function, not only for analytical convenience, but because optimizing over it will be much faster.[12]

Drawing a line between points? How hard can it be?

Leave it to graduate school to make the answer to the above question "As difficult as possible." Turns out that just like we can use GLS to fit linear regression lines, we can also use MLE.[13] To do this, we'll need to set up a log-likelihood function. Good news - Max has done this for us. Phew.

$$\log L(\mathbf{b}, \sigma^2) = -\frac{n}{2}\log(2\pi) - \frac{n}{2}\log(\sigma^2) - \frac{1}{2\sigma^2}(\mathbf{y} - \mathbf{Xb})'(\mathbf{y} - \mathbf{Xb})$$

If you're really gung-ho, you could solve for $\mathbf{b}$ and σ^2 analytically, and get $\mathbf{b_{ML}} = \mathbf{b_{OLS}} = (\mathbf{X}'\mathbf{X})^{-1}\mathbf{X}'\mathbf{y}$ and that $\sigma^2 = \frac{\mathbf{e}'\mathbf{e}}{n} \neq s^2$ (because of a degrees-of-freedom correction). But that's more work than I feel like doing right now. So let's instead use the `optim()` function, which is just like `optimize()`, but allows us to optimize over more than one parameter at once.

We'll start by generating some fake data:

[12] You could've actually skipped all of this nonsense if you'd known that it can actually be shown analytically that the best guess of p is the sample mean of our observed flips, but oh well.

[13] This might seem like overkill - but the point is just that we can use MLE to actually do useful things. The things we'll usually use it for are more complicated than this.

```
e <- rnorm(10000)
x <- rnorm(10000)
y <- 2 + 5*x + e

data <- cbind(y, 1, x) %>%
  as.tbl_df()

names(data) <- c("y", "ones", "x")
```

And we'll define our linear regression log-likelihood function. We're going to actually define this function in such a way that we feed it our actual data, rather than making it flexible, for easier interaction with the `optim()` function. Brownie points for making it fully general on your own time.

```
lmLogLike <- function(theta) {
  sigma2 <- theta[1]
  beta <- theta[-1]
  ydata <- tbldfGrabber(data, "y")
  xdata <- tbldfGrabber(data, c("ones", "x"))
  e <- ydata - xdata%*% beta
  n <- dim(data)[1]

  out <- -n/2*log(2*pi) - n/2 * log(sigma2) - 1/(2*sigma2) * t(e) %*% e
  return(-out)
}
```

And finally set up our optimization. Rather than setting a range over which to optimize, we now set parameter guesses. Note that your optimization will work better and faster if you make reasonable guesses. Note also that σ^2 is our first estimated parameter.

```
# par sets the orignal parameters, fn sets the fxn,
#   $par says grab only the estimates out
optimLM <- optim(par = c(3, 2, 1), fn = lmLogLike)$par
names(optimLM) <- c("sigma2", "ones", "x")
```

Looks good! Let's compare this to OLS:

```
olsLM <- smallOLS(data, "y", c("ones", "x"))
```

Exactly the same. Excellent. We can also calculate σ^2 easily with the following function (this is embedded in our big OLS function from last week):

```
s2 <- function(data, y, X) {
  n <- nrow(data)
  k <- length(X)
  ydata <- tbldfGrabber(data, y)
  xdata <- tbldfGrabber(data, X)
  # beta
```

```
  betahat <- smallOLS(data, y, X)
  # resids
  e <- ydata - xdata %*% betahat
  s2 <- (t(e) %*% e) / (n-k)
}
```

And let's run it:

```
sigma2 <- s2(data, "y", c("ones", "x"))
```

And compare to MLE:

```
#beta
optimLM[2:3]

##     ones        x
## 1.993912 5.013527

t(olsLM)

##       ones        x
## y 1.994085 5.013407

#sigma2
optimLM[1]

##    sigma2
## 0.9981575

sigma2

##          y
## y 0.998253
```

Pretty darn close! Unlike GLS, MLE is something you might use a little more often. Usually not to calculate linear models - but it's useful for discrete choice estimation and other structural problems.

We'll stop here for now, and pick up next week with large-sample properties of OLS.